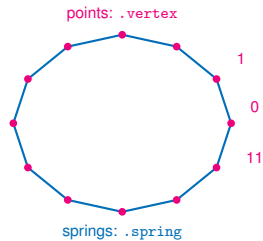
# STRUCTURE FILES

How an immersed structure is described to IBAMR: vertices, springs, beams, targets and the rest

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 6




Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

## LEARNING GOALS

By the end of this chapter you should be able to:

1. name every structure file that IBAMR's standard initializer reads, say what each one adds to the model, and read the ones used in this series (`.vertex`, `.spring`, `.beam`, `.target`) line by line;
2. say how points are numbered, what the stiffness numbers in the files mean and in what units, and what is required and what is optional in each line;
3. connect the files to the input file (`structure_names`, `level_number`, the `uniform_*` and `enable_*` keys) and to the log, which reports what was read;
4. write a generator script (MATLAB or Python) that produces the files for a new structure, and check the result before running;
5. recognize the errors IBAMR prints for a malformed file, and the mistakes it does not catch.

> Every format in this chapter was checked against the reader in IBAMR 0.19.0 (`src/IB/IBStandardInitializer.cpp`) and against runs of the examples with deliberately broken files. Where the official header comment and the code disagree, the code wins, and the slide says so.

# LEARNING IBAMR: WHERE TO LOOK THINGS UP

- The code and its examples: https://github.com/IBAMR/IBAMR. The examples folder is the best reference for what each file and key does in practice (chapter 15).
- The web site and documentation index: https://ibamr.github.io and https://ibamr.github.io/docs. The old per-class pages (`ibamr.github.io/IBAMR/ibamr/html/`) no longer exist.
- The file formats are documented in the header `include/ibamr/IBStandardInitializer.h` of the source tree, which autoibamr leaves unpacked under `tmp/unpack/IBAMR-<version>/`. Open it next to this chapter.

> When a slide here and the header comment disagree, trust the slide: the comment says the rest length of a spring is optional, and the reader stops with an error when it is missing (slide 12, tested).

# LEARNING IBAMR: THE COMMUNITY AND THE METHOD

- The wiki and its FAQ: https://github.com/IBAMR/IBAMR/wiki (partly dated: it predates CMake builds).
- Questions: the IBAMR Users Google group, https://groups.google.com/g/ibamr-users, and GitHub Discussions, https://github.com/IBAMR/IBAMR/discussions. Development discussion is on the developers' group, `ibamr-dev@googlegroups.com`.
- The method: Peskin's page, https://math.nyu.edu/~peskin/, and the 2002 Acta Numerica paper (chapter 5).

The 2015 lecture pointed to the Google Code site (`ibamr.googlecode.com`) for the class documentation, the source browser and the file-format page; Google Code closed in 2016. The 2020 lecture's class-documentation link returns 404 today. Links checked October 2026.

# TWO KINDS OF INPUT

The input file, `input2d` or `input3d` (chapters 3 and 7):

- physical parameters: density, viscosity, the box, boundary conditions;
- numerical parameters: the grid levels, `DT`, the run length, output intervals;
- and the block that names the structures.

The structure files, one set per structure, named `<name>.<extension>`:

- the initial positions of the points (`.vertex`), the only required file;
- the elastic elements that join or hold them: springs, beams, target points, and the less common ones.

> The structure files say what the structure *is*; the input file says where it goes, on which grid level, and lets you override its stiffnesses without rewriting it (slide 24). The program (`main.cpp`) usually knows nothing about either: it reads both at start-up through `IBStandardInitializer`.

The files are plain text. You write them with a script (slides 29–33) and read them with any editor; the examples in this series ship both the files and the scripts that made them.

# THE FILES IBAMR READS

| File | What it specifies | Required? | Used in |
|---|---|---|---|
| `.vertex` | the initial position of every point | yes | every example |
| `.spring` | linear or nonlinear springs between pairs of points | no | IB/01–04, 06, 07 |
| `.beam` | bending resistance for triples of points, with an optional preferred curvature | no | IB/02, 03, 07; chapter 10 |
| `.target` | points tethered to their initial position (or to a moving target) by stiff springs | no | IB/02–04, 07, 08; chapter 9a |
| `.xspring` | “crosslink” springs that may join points of different structures (global indices) | no | chapter 9a |
| `.anchor` | points fixed in place that exchange nothing with the fluid | no | slide 18 |
| `.mass` | extra mass on points (penalty immersed boundary) | no | chapter 13 |
| `.rod`, `.director` | Kirchhoff rods that resist bending and twisting, and their material frames | no | slide 21; chapter 15 |
| `.inst` | flow meters built from points | no | slide 22; chapter 15 |
| `.source` | fluid sources and sinks attached to points | no | slide 22; chapter 15 |

The extensions are fixed; the base name is yours and goes in `structure_names`.

# WHAT THE TABLE DOES NOT SAY

- Every optional file that is absent is silently skipped, with one line in the log (slide 25). A structure with no `.spring`, `.beam` or `.target` file is a cloud of free points that the fluid carries along, and nothing on the screen says so.
- The files describe elements, not materials: a spring is two points and two numbers, a beam three points and one number. Chapters 8a and 8b say how to choose the numbers from a material and a size; chapter 9b how to make them depend on time.
- One structure can have all of the files at once (the leaf has four), and the same point can be in several elements: a point can be the end of two springs, the middle of a beam and a target point, as the leaf's points 50 and 51 are.
- IBFE structures are different: a finite element mesh file (MSH or ExodusII) replaces this whole set, and the elasticity is a function in the program (Part IV). The two can be mixed in one run.

# HOW IBAMR FINDS THE FILES

```
IBStandardInitializer {
   max_levels = MAX_LEVELS
   structure_names = "leaf_512"
   leaf_512 {
      level_number = MAX_LEVELS - 1
   }
}
```

Several structures: `structure_names = "plate2d_left_512", "plate2d_rght_512"`, each with a sub-block (IB/07; chapter 9a).

- `structure_names` lists the base names; for each, IBAMR opens `<name>.vertex` in the working directory (where you run `mpirun`), then tries every other extension.
- Each structure has its own sub-block with its `level_number`: the grid level it lives on, almost always the finest, `MAX_LEVELS - 1` (chapter 5).
- The program never names the files: `main.cpp` only passes the `IBStandardInitializer` block to the initializer.

> The base name is also the name of the mesh in VisIt (`leaf_512_mesh`) and the prefix of the restart data, so choose it before you generate the files and keep it. The older key `base_filenames`, one list per level, still works but is not used in this series.

# THE .VERTEX FILE

```
N                 # number of points in the file
x_0  y_0          # position of point 0
x_1  y_1          # position of point 1
...
x_{N-1}  y_{N-1}  # position of point N-1
```

In 3D each line has three numbers, `x y z`.

The rubber band's file begins

```
644
0.0000000000000000e+00 1.0000000000000001e-01
1.0244161346221071e-03 9.9995240574191152e-02
2.0487347565924115e-03 9.9980962749807278e-02
3.0728583625411962e-03 9.9957167885933298e-02
```

and continues for 644 lines: 644 points of the band, in meters, each about $\Delta x/2$ from the next.

- The first line is the count $N$; then exactly $N$ lines of coordinates, in the units of the input file (meters here).
- The index of a point is its line number, counting from 0: the first coordinate line is point 0, the last is point $N-1$. Nothing in the file says "point 7"; its place does.

# READING A `.vertex` FILE

- Text after a `#`, `%` or `!` on a coordinate line is ignored, so you may annotate a `.vertex` file. A line that holds only a comment is an error (slide 26).
- `posn_shift` and `length_scale_factor` in the structure's input sub-block move and scale the points as they are read, without rewriting the file (slide 24).
- The order of the lines is the geometry's own order: for a curve, walk along it; for a ring, go around once; for several rings (the bristles of IB/04), one ring after another. The `.spring` and `.beam` files refer to these line numbers, so the generator that writes the points must be the one that writes the elements.
- Coordinates are written with 17 significant digits (`%1.16e`) so that the file holds the exact double; three decimals would do for the physics but would make the Python and MATLAB generators disagree on the last digit (Appendix A compares them).
- A 3D `.vertex` file for a 2D run, or the reverse, fails at once: the reader expects exactly `NDIM` numbers per line.

# POINT NUMBERS: LOCAL AND GLOBAL

- Within one structure, points are numbered 0 to $N-1$ by their order in the `.vertex` file, and the `.spring`, `.beam`, `.target`, `.anchor` and `.mass` files refer to these local numbers.
- When the input lists several structures, IBAMR numbers all their points together in the order of `structure_names`: the first structure's points keep their numbers, the second structure's are offset by the first's count, and so on. These are the global indices.
- Only one file in the standard set uses global indices: `.xspring`, because a crosslink spring may join two structures. The program can ask for a structure's range with `getLagrangianStructureIndexRange` (IB/07).
- Changing the order in `structure_names` changes every global index; the local files do not change.

> IBAMR counts from 0; MATLAB, and IB2d's MATLAB version, count from 1 (IB2d's Python version counts from 0: chapter 4a). A `.spring` file written with MATLAB's loop variable as the index is off by one, and IBAMR stops at the first index equal to $N$: `vertex index 644 is out of range` (slide 26). The generators in this series loop `for s = 0:npts-1` for exactly this reason.

Chapter 9a uses local and global indices side by side for the two plates of IB/07 (points 0–102 and 103–205).

# THE .SPRING FILE

```
M                              # number of springs
i_0  j_0  kappa_0  L_0  [fcn_0  [p_1 p_2 ...]]
i_1  j_1  kappa_1  L_1  [fcn_1  [...]]
...
```

Each line: the two points it joins, its stiffness $\kappa$, its rest length $L$, and optionally a force-function index (slide 14) and extra parameters for that function.

The rubber band's file:

```
644
0 1 2.0499156670236122e+06 9.7564989241919039e-04
1 2 2.0499156670236122e+06 9.7564989241919039e-04
...
642 643 2.0499156670236122e+06 9.7564989241919039e-04
643 0 2.0499156670236122e+06 9.7564989241919039e-04
```

644 springs for 644 points: each joins a point to the next, and the last one closes the ring.

The force of a spring of current length $R$ is $\kappa\,(R - L)$ along the line between its points, pulling when stretched and pushing when compressed, with equal and opposite forces on the two ends (`default_spring_force` in `IBSpringForceFunctions.h`).

# SPRINGS: WHAT ELSE TO KNOW

- Any number of springs may meet at a point; a point may also have none. A network of springs (a sheet, a lattice, the cross-braced wing of chapter 9a) is just a longer `.spring` file.
- The lower of the two indices is the spring's "master" point, the one whose data holds the spring; IB/06 changes rest lengths through it.
- A spring with $\kappa = 0$ does nothing but is still drawn by VisIt (slide 35): the header suggests a zero-stiffness `.spring` file for structures that have no springs, so that they can be plotted as lines.
- A spring whose rest length is shorter than the points' distance starts under tension; longer, under compression. The rubber band starts stretched (an ellipse with the rest lengths of a circle), which is what makes it move.
- Springs do not resist bending: a chain of springs folds freely, as IB/06 shows. Beams add that (slide 16).

# WHAT IS REQUIRED ON A SPRING LINE

- The header comment in `IBStandardInitializer.h` says the rest length and the function index are optional, with defaults 0 and 0.
- The rest length is not optional: a file with only `i j kappa` on each line stops IBAMR with `Invalid entry in input file encountered on line 2 of file rubber_band_512.spring` (tested with 0.19.0). The function index is optional.
- Four numbers per line, then: two indices, $\kappa$, $L$. Anything after the required and optional numbers is ignored, which is why a trailing `// spring` on each line passes (slide 26). Do not rely on it.

> If you want a zero rest length (a spring that pulls its ends together until they meet), write the 0: `i j kappa 0.0`.

# THE CLOSING SPRING

- A ring of $N$ points needs $N$ springs, the last one joining point $N-1$ back to point 0.
- The UNC course's generator wrote `npts-1` springs with the loop `for s = 0:npts-2`, so its band had a gap: 643 springs for 644 points. The 2022 version and IB/01 write 644, with `mod(s+1, npts)` as the second index of the last spring.
- The 2018 and 2020 lectures showed the 643-spring file as the example output; the gap went unnoticed for years because the band still looked round.

> A missing closing spring is invisible in the files and in the log (644 points, 643 edges: both numbers look fine). It shows in VisIt as a gap in the mesh line, and in the physics as a band that does not hold its pressure. Check the count: a closed curve has as many springs as points.

# SPRING FORCE FUNCTIONS

The fifth number on a spring line, the function index, chooses how the force depends on the length. Index 0 (the default) is the linear spring $F = \kappa\,(R - L)$. A program can register its own functions under other indices:

```
// in main.cpp, before the run (chapter 9b)
double my_force(double R, const double* params, int lag_mastr_idx, int lag_slave_idx)
{   // params[0] = kappa, params[1] = L, params[2..] = the extras on the spring's line
    return params[0] * (R - params[1]) * (R - params[1]);
}
...
ib_force_fcn->registerSpringForceFunction(1, &my_force);
```

A line `i j kappa L 1 a b` in the `.spring` file then uses `my_force` with `params = {kappa, L, a, b}`.

# SPRING FORCE FUNCTIONS: HOW THEY ARE USED

- The extra parameters are read from the line after the index, as many as there are (`while (line_stream >> param)`), and stored with the spring.
- `uniform_spring_force_fcn_idx` in the input sub-block sets the index for every spring of a structure without rewriting the file (slide 24).
- The function receives the two point indices too, so it can look up anything about them (a stiffness that varies along the body, say). Register functions before the time loop starts.
- A derivative function can be registered alongside (`registerSpringForceFunction(idx, &f, &df)`); IBAMR uses it for the implicit solvers and ignores it in the explicit ones used here.
- Chapter 9b builds a muscle-like spring this way; IB/06 changes the rest length of linear springs in time instead, which needs no new function.

# WHAT THE STIFFNESS NUMBERS MEAN, AND THEIR UNITS

The generators of this series write

$$\kappa_{\text{file}} = \frac{K}{\Delta s} \quad \text{(springs and targets)}, \qquad \kappa_{b,\text{file}} = \frac{EI}{\Delta s^3} \quad \text{(beams)},$$

where $\Delta s$ is the point spacing, $K$ is the structure's stretching stiffness (its tension per unit strain, in newtons) and $EI$ its bending stiffness (in N m$^2$). Why:

- A spring between neighbors $\Delta s$ apart, stretched by $R - L = \varepsilon\,\Delta s$ (strain $\varepsilon$), pulls with $\kappa_{\text{file}}(R - L) = K\varepsilon$: the tension of the material, independent of how finely it is discretized.
- A beam's force uses the second difference $X_{i-1} - 2X_i + X_{i+1} \approx \Delta s^2\,\partial_s^2 X$; dividing $EI$ by $\Delta s^3$ makes the bending moment per point come out as $EI$ times the curvature, again independent of the spacing.
- A target point's force $\kappa_{\text{file}}(X_{\text{target}} - X)$ is then the force per point of a tether with stiffness $K$ per unit length.

# THE STIFFNESS NUMBERS: CONSEQUENCES

- The file numbers change when you refine the structure, and that is why the generators, not the files, hold the physical constants: `kappa_spring = 2.0e3` (N) in IB/01's script becomes $2.05 \times 10^6$ N/m in the file, for $\Delta s = 9.8 \times 10^{-4}$ m.
- Two structures with the same $K$ but different point spacings have different file numbers; comparing stiffnesses across files is meaningless without $\Delta s$.
- The older lectures never said what the file $\kappa$ was; the 2013 generator comments called $\kappa_{\text{spring}}$ a "spring constant (Newton)" without the division. The expression they used, `kappa_spring*ds/(ds^2)`, is $K/\Delta s$ written in two steps.
- In dimensionless inputs (the IBFE examples, most of IB2d's) the same scaling holds with dimensionless $K$ and $EI$. Chapter 8b shows how to choose $K$ and $EI$ from a real material, and how they scale.

> Rule: keep $K$ and $EI$ in the generator, compute $\Delta s$ from the grid, and let the script do the division.

# THE .BEAM FILE

```
M                                   # number of beams
i_0  j_0  k_0  kappa_0  [Cx_0  Cy_0  [Cz_0]]
i_1  j_1  k_1  kappa_1  [Cx_1  Cy_1  [Cz_1]]
...
```

Each line: three consecutive points, a bending rigidity $\kappa_b$, and optionally a preferred curvature vector $\mathbf{C}$ with as many components as the problem has dimensions.

The leaf's file (IB/03):

```
101
0 1 2 5.3060400000000009e+06
1 2 3 5.3060400000000009e+06
...
100 101 102 5.3060400000000009e+06
```

101 beams for 103 points: one per interior point.

The beam resists the second difference $\mathbf{X}_i - 2\mathbf{X}_j + \mathbf{X}_k$: its energy is $\frac{1}{2}\kappa_b \left|\mathbf{X}_i - 2\mathbf{X}_j + \mathbf{X}_k - \mathbf{C}\right|^2$, and the force on each of the three points is minus the gradient of that energy, with the stencil $(1, -2, 1)$.

# BEAMS: WHAT ELSE TO KNOW

- The middle index $j$ is the master point and must be an interior point of the triple; $N$ points in a line give $N - 2$ beams, one per interior point, and a ring of $N$ points gives $N$ beams (the two that wrap around included).
- With no curvature given, $\mathbf{C} = 0$: the beam is straight at rest. A curvature vector makes a curved rest shape, which is how chapter 10 makes a jellyfish bell and changes it in time; the vector is the preferred value of $\mathbf{X}_i - 2\mathbf{X}_j + \mathbf{X}_k$, so it scales with $\Delta s^2$.
- Beams and springs on the same points are the usual pair: springs keep the length, beams the shape. The leaf has both; the bristles of IB/04 have neither, only targets.
- `uniform_beam_bend_rigidity` and `uniform_beam_curvature` override the file (slide 24).
- Beams are the fourth file that VisIt does not draw (slide 35): a structure with beams only shows as points.

# THE .TARGET FILE

```
M                    # number of target points
i_0  kappa_0  [eta_0]
i_1  kappa_1  [eta_1]
...
```

Each line: a point index, the tether stiffness $\kappa$, and optionally a damping coefficient $\eta$ (0 if absent).

The leaf's two target points, and the first lines of the bristles' file, where every point is a target:

```
2
50 3.0600000000000004e+04
51 3.0600000000000004e+04
```

```
297
0 4.0960000000000000e+06
1 4.0960000000000000e+06
```

A target point is tethered to a target position by a linear spring of zero rest length and, if $\eta > 0$, a dashpot: force $\kappa\,(\mathbf{X}_{\text{target}} - \mathbf{X}) - \eta\,\mathbf{U}$. The target position starts at the point's initial position.

# TARGET POINTS: WHAT ELSE TO KNOW

- A target point approximately pins its point: the stiffer $\kappa$, the smaller the lag, and the smaller the stable time step (chapter 5, slide 27). The lag is the force divided by $\kappa$, which the READMEs of IB/03, 04 and 07 report.
- The program may move the target positions during the run, which is how prescribed motion is done (IB/04, IB/07, IB/08; chapter 9a). The file only says which points are tethered and how stiffly; the motion is code.
- The damping $\eta$ dissipates the oscillation of a point about its target; the 2013 lecture lost its symbol in a font change, and the course files never used it. With $\eta = 0$ the target is a pure spring, a Kelvin–Voigt element with the dashpot removed.
- Listing a point twice gives the warning `Duplicate target point node`; `uniform_target_stiffness` and `uniform_target_damping` override the file (slide 24).
- Targets are not drawn by VisIt either, but the points they hold are: a tethered structure looks like any other.

# THE .ANCHOR FILE

```
M        # number of anchor points
i_0
i_1
...
```

One index per line.

An anchor point neither spreads force to the fluid nor interpolates velocity from it: it is literally anchored where it starts, whatever the fluid does.

- Anchor points are not target points. A target point is tethered by a spring and moves a little; an anchor point does not move at all and exerts no force of its own. A spring from a structure to an anchor point behaves like a spring to a fixed wall.
- Use them for the fixed end of a tether, for a support that should not feel the flow, or to hold the end of a filament exactly.
- `enable_anchor_points = FALSE` in the input sub-block turns them back into ordinary points without editing the file.

> Think of target points as tethered to invisible anchor points: the 2020 lecture's phrase, and a good way to keep the two apart.

# THE .MASS FILE

```
M                    # number of mass points
i_0  mass_0  kappa_0
i_1  mass_1  kappa_1
...
```

Each line: a point index, the mass carried by that point, and the penalty stiffness of the spring that ties the point to its massive copy.

- The immersed boundary points themselves are massless: they move with the fluid.
- To give a structure mass, IBAMR's penalty immersed boundary method attaches to each listed point a "ghost" particle with the given mass, which moves by Newton's laws (gravity included), and ties the two with a stiff spring of zero rest length.

# MASS POINTS: WHAT ELSE TO KNOW

- The penalty stiffness plays the same role as a target stiffness: stiffer means the point follows its massive copy more closely and the time step must be smaller.
- The mass is per point: a structure of $N$ points with mass $m$ each weighs $Nm$, so when you refine the structure, divide the mass per point as you multiply the points (the generators do this with $\Delta s$, as for the stiffnesses).
- `uniform_bdry_mass` and `uniform_bdry_mass_stiffness` override the file; `enable_bdry_mass = FALSE` ignores it.
- A `.mass` file alone does nothing: the penalty method has to be switched on in the input and in `main.cpp` (the integrator reads the mass data only then). Without that, the file is read, the log reports it, and the structure stays massless.

> Chapter 13 (adding mass) runs the 2020 example with a `.mass` file and the input keys that switch the penalty method on, and shows what a heavy and a buoyant structure do.

# THE .XSPRING FILE

```
M
i_0  j_0  kappa_0  L_0  [fcn_0 ...]
...
# i and j are GLOBAL indices
```

Crosslink springs have the same line format as `.spring`, but the indices are global (slide 8), so a crosslink spring may join points of two different structures.

- The two structures must be on the same grid level.
- The lower index is the master point, as for springs; the same force functions and the same optional columns apply, and `uniform_xspring_stiffness`, `uniform_xspring_rest_length` and `uniform_xspring_force_fcn_idx` override the file.
- They are the file route for tying two structures together (two plates hinged at a point; a wing attached to a body); chapter 9a lists them with the other ways, and none of this series' examples uses one.
- Reordering `structure_names` silently rewires every crosslink spring: regenerate the file when you do.

# THE .ROD AND .DIRECTOR FILES

Rods: Kirchhoff rods that resist bending *and twisting*, for filaments such as flagella. Each line joins two consecutive points and carries ten material numbers:

```
M
i  j  ds  a1 a2 a3  b1 b2 b3  kappa1 kappa2 tau
```

`ds` is the rest length; `a1 a2 a3` the bending and twisting moduli; `b1 b2 b3` the shear and stretching moduli; `kappa1 kappa2 tau` the intrinsic curvatures and twist.

- A rod needs a material frame at every point: the `.director` file gives three unit vectors per point, one per line, so $3N$ lines of three numbers after the count. IBAMR warns if a vector is not of unit length.
- The force generator for rods is `IBKirchhoffRodForceGen`, registered in `main.cpp` instead of (or besides) the standard one; the upstream examples that use rods are listed in chapter 15.
- `uniform_rod_properties` overrides all ten numbers at once.

> None of the examples in this series uses rods. They are here so that the file set is complete, and because the error messages for a malformed `.rod` file look like those for springs.

# THE .INST FILE

```
M_meters
name_0
name_1
...
N_points
i  meter_idx  node_idx
...
```

Instrumentation: flow meters. The file names a number of meters, then lists points with the meter and the position along the meter they belong to; IBAMR integrates the flow through the surface the points define and the pressure on it, and writes them during the run.

- Read only when `enable_instrumentation = TRUE` for the structure (the default is `FALSE`), and only useful with an `IBInstrumentPanel` in `main.cpp`.
- Chapter 11 measures forces and flow rates without meters, from the output files; chapter 15 points to the upstream example with meters.

# THE .SOURCE FILE

```
M_sources
name_0
...
r_0        # radius of each source
...
N_points
i  source_idx
...
```

Sources and sinks: fluid sources attached to points, each with a name and a radius; the program sets their strengths in time (an inflow through a valve, a jet).

- Read only when `enable_sources = TRUE` (default `FALSE`), and needs an `IBStandardSourceGen` in `main.cpp` that returns the strength of each source at each time.
- The radius must be positive: `r <= 0` is rejected when the file is read.
- Chapter 15 points to the upstream example that uses sources; none of this series' examples does.

> The layouts of these two files are reconstructed from the readers in the 0.19.0 source; the header comment has the authoritative version with the meaning of each field.

# COMMENTS, LINE ENDINGS, NUMBERS: THE RULES, TESTED

| Rule | What IBAMR 0.19.0 does |
|---|---|
| The first line is the count | It must be a single integer. A missing count makes the reader take the first data line as the count: `Invalid entry in input file encountered on line 1`. |
| Then exactly "count" data lines | Fewer: `Premature end to input file encountered before line` $k$. More: the extra lines are ignored. |
| Comment lines | A line that is only a comment is an error (it is read as data). Comments *after* the numbers are allowed in `.vertex` files (`#`, `%` or `!` start the comment). |
| Trailing text in the other files | Ignored once the required and optional numbers have been read; a `.spring` line ending in `// spring` passes. Do not count on it. |
| Windows line endings (CRLF) | Accepted: the readers parse numbers and ignore the carriage return. (The input file is *not* so forgiving: chapter 3.) |
| Blank lines | An error where a data line is expected. |
| Number format | Anything C++ reads as a double: `0.1`, `1e-3`, `9.7564989241919039e-04`. The generators write `%1.16e` so that the files round-trip exactly. |
| Separators | Spaces or tabs, any number of them. No commas. |
| Order of lines | Free, except in `.vertex` (the line number is the index) and `.director` (three lines per point, in point order). |

Each row was checked by running the rubber band with the corresponding change to its files (the test script is in the working folder of these notes).

# OVERRIDING A FILE FROM THE INPUT

```
IBStandardInitializer {
   max_levels = MAX_LEVELS
   structure_names = "leaf_512"
   leaf_512 {
      level_number = MAX_LEVELS - 1
      // optional, per structure:
      uniform_spring_stiffness   = 3.06e4
      uniform_spring_rest_length = 9.8e-4
      uniform_beam_bend_rigidity = 5.31e6
      uniform_target_stiffness   = 3.06e4
      uniform_target_damping     = 0.0
   }
}
```

- `uniform_*` keys replace the corresponding column of a file by one value for the whole structure: stiffness, rest length and function index for springs and crosslink springs; rigidity and curvature for beams; stiffness and damping for targets; mass and stiffness for mass points; all rod properties.
- Handy for a stiffness sweep (chapter 3's job array): one input file per value, the same structure files for all.
- The file must still exist and parse: the override replaces values, it does not create elements.

# SWITCHING OFF, MOVING AND SCALING FROM THE INPUT

```
leaf_512 {
   level_number = MAX_LEVELS - 1
   enable_beams          = FALSE   // ignore leaf_512.beam
   enable_target_points = TRUE    // the default
   posn_shift            = 0.1, 0.0
   length_scale_factor  = 0.5
   use_file_batons       = FALSE
}
```

- `enable_springs`, `enable_beams`, `enable_target_points`, `enable_anchor_points`, `enable_bdry_mass`, `enable_xsprings`, `enable_rods`, `enable_instrumentation`, `enable_sources`: `FALSE` ignores that file. All default to `TRUE` except the last two.
- `posn_shift` moves every point as it is read; `length_scale_factor` scales positions *and* spring rest lengths (but not stiffnesses, which you would rescale by hand, slide 15).
- `use_file_batons = TRUE` makes the processes read the files one after another, for very large files on many cores.

These keys let one set of files serve several runs: the same leaf, moved, scaled, with or without its beams.

# WHAT THE LOG TELLS YOU

```
IBStandardInitializer:  Reading from input files.
IBStandardInitializer:  processing vertex data from ASCII input file named rubber_band_512.vertex
IBStandardInitializer:  read 644 vertices from ASCII input file named rubber_band_512.vertex
IBStandardInitializer:  processing spring data from ASCII input file named rubber_band_512.spring
IBStandardInitializer:  read 644 edges from ASCII input file named rubber_band_512.spring
IBStandardInitializer:   file rubber_band_512.xspring on MPI process 0 does not exist: skipping read.
IBStandardInitializer:   file rubber_band_512.beam on MPI process 0 does not exist: skipping read.
IBStandardInitializer:   file rubber_band_512.target on MPI process 0 does not exist: skipping read.
...
```

- These lines are at the top of the log file (`IB2d.log` or `IB3d.log`), not on the screen. Read them after every change to a structure.
- Check the counts: 644 vertices and 644 edges for a closed band; 103 vertices, 102 springs, 101 beams and 2 target points for the leaf.
- A skipped file you meant to supply (a typo in the name, a file in the wrong folder) is reported only here. The run goes on without it: a band without springs is just 644 free points, and nothing warns you.

# THE WARNINGS WORTH SEARCHING FOR

```
grep -i warning IB2d.log
```

- `Spring with zero spring constant`, `Beam with zero bending rigidity`, `Target point with zero penalty spring constant`: a column of zeros, usually a generator bug (a variable used before it was set, or a unit mix-up that produced 0).
- `Duplicate target point node`, `Duplicate anchor point node`, `Duplicate boundary mass point node`: a point listed twice; the last entry wins.
- `Director vector on line ... does not have unit length`: the `.director` vectors are normalized by IBAMR after the warning.
- `Potentially invalid entry in input file` for a rod whose curvature or twist could not be read: the rod gets the default and the run continues.

> Warnings do not stop the run. The ones above all mean that the structure in the run is not the one you think you wrote.

# WHAT CAN GO WRONG: ERRORS THAT STOP THE RUN

| Mistake (tested on the rubber band, IBAMR 0.19.0) | What IBAMR prints, and the fix |
|---|---|
| A comment line at the top of `.vertex` | `Invalid entry in input file encountered on line 1 of file rubber_band_512.vertex`. Put comments after the numbers, or nowhere. |
| The count line missing from `.spring` | `Invalid entry in input file encountered on line 1 of file rubber_band_512.spring`. Add the count. |
| Indices starting at 1 | `Invalid entry in input file encountered on line 644 of file rubber_band_512.spring`, then `vertex index 644 is out of range`. Subtract 1 in the generator. |
| Fewer spring lines than the count | `Premature end to input file encountered before line 601`. Fix the count or the loop. |
| A vertex count larger than the file | `Premature end to input file encountered before line 646 of file rubber_band_512.vertex`. |
| No rest length (three columns in `.spring`) | `Invalid entry in input file encountered on line 2`. The rest length is required (slide 11). |
| No `.vertex` file | `Cannot find required vertex file: rubber_band_512.vertex`. Check the name in `structure_names` and the working directory. |

Every error above stops the run before the first time step, so a malformed file costs seconds, not hours: run a short test after generating files (slide 34). The process exits with code 134 and an `Aborting` line after the message.

# WHAT CAN GO WRONG: MISTAKES IBAMR DOES NOT CATCH

| Mistake (tested) | What happens, and how to see it |
|---|---|
| No `.spring` file | No error. The log says `skipping read`; the band has no elasticity and drifts with the fluid. Check the log (slide 25). |
| A missing closing spring | No error. 643 edges in the log; a gap in VisIt; a band that leaks pressure (slide 12). |
| CRLF line endings | No error; the files read correctly. Convert them anyway, for the input file's sake. |
| A trailing comment in `.spring` after the numbers | No error; ignored. A comment in the middle of a line is not: the numbers after it are lost. |
| Stiffness written without the division by $\Delta s$ | No error; a structure a thousand times too soft or too stiff (slide 15). Compare with an example's file. |
| The structure outside the box, or on a wall | No error from the reader; the run starts and misbehaves (chapter 7). Look at the structure in VisIt before the long run. |
| Points of two structures on top of each other | No error; the two share the same fluid cells (chapter 5, slide 16). |

The second kind is the expensive kind: the run completes, and only the pictures or the forces are wrong. The checks on slides 34 and 35 catch most of them in a minute.

# WRITING THE FILES: THE GENERATOR PATTERN

```
% generate_mesh2d.m (IB/01): the constants
L = 1;               % length of computational domain (m)
N = 512;             % Cartesian grid cells at the finest level
dx = L/N;            % Cartesian mesh width (m)

radius = 0.1;        % radius of band (m)
epsilon = 0.02;      % deformation in the x-direction
band_length = 2*pi*radius;           % rubber band length (m)
npts = ceil(2*(band_length/L)*N);    % points along the band: 644
ds = band_length/npts;               % point spacing (m)
dtheta = 2*pi/npts;

mesh_name = 'rubber_band_';          % structure name

kappa_spring = 2.0e3;   % stretching stiffness K (N)
```

The first half of every generator: the grid, the shape, the physical constants, and the name. Nothing is written yet.

# THE GENERATOR PATTERN: THE FIVE PARTS

Every generator in this series has the same shape:

1. the grid: $L$, $N$ and $\Delta x = L/N$ at the finest level, so that the point spacing follows the grid (chapter 5);
2. the geometry: a shape, its size, and the number of points, $\lceil 2\,\ell/\Delta x \rceil$ for a curve of length $\ell$;
3. the physical constants, $K$ and $EI$, in physical units;
4. one loop per file, writing the count and then the lines with `%1.16e`;
5. the structure name shared by all the files.

> Change $N$ and everything follows: more points, closer together, with the file stiffnesses rescaled (slide 15). That is the whole reason to keep the generator with the example.

# WRITING THE FILES: THE VERTEX LOOP

```
% Write out the vertex information
vertex_fid = fopen([mesh_name num2str(N) '.vertex'], 'w');
fprintf(vertex_fid, '%d\n', npts);
for s = 0:npts-1
   X(1) = (radius+epsilon)*sin(s*dtheta);
   X(2) = radius*cos(s*dtheta);
   fprintf(vertex_fid, '%1.16e %1.16e\n', X(1), X(2));
end
fclose(vertex_fid);
```

- The file name is the structure name plus $N$: `rubber_band_512.vertex`.
- The count goes first, then one line per point.
- The loop variable `s` runs from 0, and is the point index. MATLAB's own arrays start at 1, but the file does not care what MATLAB calls things.
- `%1.16e` prints 17 significant digits: the file reproduces the double exactly, and the Python version writes the same bytes (slide 31).

# WRITING THE FILES: THE SPRING LOOP

```
% Write out the spring information
spring_fid = fopen([mesh_name num2str(N) '.spring'], 'w');
fprintf(spring_fid, '%d\n', npts);
for s = 0:npts-1
   fprintf(spring_fid, '%d %d %1.16e %1.16e\n', ...
       s, mod(s+1,npts), kappa_spring*ds/(ds^2), ds);
end
fclose(spring_fid);
```

- `mod(s+1, npts)` closes the ring: the last spring joins point 643 to point 0.
- `kappa_spring*ds/(ds^2)` is $K/\Delta s$ written the way the course scripts wrote it (a stiffness per unit length, times $\Delta s$, divided by $\Delta s^2$).
- The rest length is the undeformed spacing of a circle of radius 0.1 m, so the stretched band starts under tension.
- The beam and target loops of IB/03 and IB/07 look the same, with three indices and $EI/\Delta s^3$, or one index and $K/\Delta s$.

# THE SAME GENERATOR IN PYTHON

```
# generate_mesh2d.py (IB/01), standard library only
import math
L, N = 1.0, 512
radius, epsilon = 0.1, 0.02
band_length = 2 * math.pi * radius
npts = math.ceil(2 * (band_length / L) * N)
ds = band_length / npts
dtheta = 2 * math.pi / npts
kappa_spring = 2.0e3
name = "rubber_band_%d" % N

with open(name + ".vertex", "w", newline="\n") as f:
    f.write("%d\n" % npts)
    for s in range(npts):
        x = (radius + epsilon) * math.sin(s * dtheta)
        y = radius * math.cos(s * dtheta)
        f.write("%1.16e %1.16e\n" % (x, y))

with open(name + ".spring", "w", newline="\n") as f:
    f.write("%d\n" % npts)
    for s in range(npts):
        f.write("%d %d %1.16e %1.16e\n" % (s, (s + 1) % npts, kappa_spring * ds / ds**2, ds))
```

# THE PYTHON GENERATOR: NOTES

- `newline="\n"` keeps Unix line endings on Windows. IBAMR would accept CRLF in these files (slide 23), but the input file would not, and mixed endings confuse `diff`.
- `%1.16e` is the same format in both languages. The shipped `.spring` file is byte-identical from either; in the `.vertex` file a few dozen coordinates differ in the 17th digit between MATLAB and Python, because their sine and cosine round differently. Appendix A lists every script and what was compared.
- `python3 generate_mesh2d.py` in the example folder rewrites the files; so does `generate_mesh2d` in MATLAB or Octave. Every generator of this series was checked both ways against its shipped files (`tools/compare_outputs.py` in the working folder of these notes).
- The Python versions need nothing beyond the standard library (NumPy only where arrays are unavoidable), so they run on Puma's system Python without any installation.

> Prefer the script that you can run: both produce files IBAMR reads identically.

# GENERATORS WITH MORE FILES: THE LEAF AND THE PLATES

IB/03's `generate_mesh2d.m` writes four files for one structure:

- `.vertex`: 103 points on $x = 0$ from $y = 0.05$ down to $-0.05$, $\Delta s = 0.1/102$;
- `.spring`: 102 springs $(s, s+1)$ with $K/\Delta s$ and rest length $\Delta s$;
- `.beam`: 101 beams $(s-1, s, s+1)$ with $EI/\Delta s^3$ and no curvature;
- `.target`: the two middle points, 50 and 51, with $K/\Delta s$.

IB/07's writes two structures with the same loops and an `offset` of $\pm 2.5$ mm, eight files in all, each with local indices 0–102; the input file's `structure_names` puts them in order and the program finds their global ranges.

IB/04 had no generator in the course (the files were inherited), so one was written in 2026 that reproduces them: nine rings of 33 points, every point a target. IB/08's `generate_mesh3d.m` writes 160 rings of 85 points in 3D, 13,600 lines, and a `.target` file for all of them.

> A structure's generator is its documentation. When you inherit files without one, write the generator first (as for IB/04) and check that it reproduces the files to the last digit; then change what you need.

# CHECKING A STRUCTURE BEFORE YOU RUN: THE FILES

```
wc -l leaf_512.vertex leaf_512.spring leaf_512.beam     # count + 1 lines each
head -3 leaf_512.vertex                                  # the count and the first points
awk 'NR>1 {print $1; print $2}' leaf_512.spring | \
    sort -n | uniq -c | sort -rn | head                  # which points carry the most springs
```

Line counts, the first lines, and the spring count per point (2 for an interior point of a chain, 1 at the ends, more at a junction). Then plot:

```
python3 test_ib_geometry.py leaf_512 --ibamr   # NAME.vertex and NAME.spring, indices from 0
```

```
test_IB2d_IBAMR_Geometry   % MATLAB or Octave: set the file names at the top of the script
```

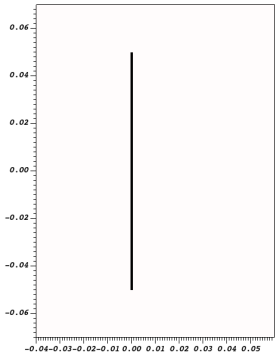
Both (in `examples/support/matlab-exercises`, Appendix A) draw the points with their index numbers and the springs between them in alternating colors, so that a missing one shows; the Python version takes the structure name and `--ibamr` for 0-based indices, the MATLAB version reads the names written at its top.

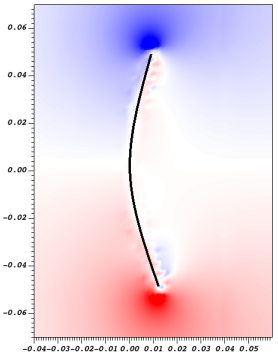
# CHECKING A STRUCTURE BEFORE YOU RUN: A SHORT RUN

- A short run is the last check: set `END_TIME` to a few steps, run, and read the log's "read N" lines (slide 25). All the file errors of slide 26 appear before the first step.
- Open `lag_data.visit` in VisIt (next slide) and look at the structure on the grid: is it where you meant, is it closed, is it on the finest level, is the spacing right?
- For a structure with no springs, add a `.spring` file with $\kappa = 0$ so that VisIt can draw it as lines; or plot the points as a point mesh.
- Look at `integral{F}` on the screen at the first steps: for a structure at rest it should be near zero; a large value at step 0 means an element is far from its rest state (a wrong rest length, a wrong curvature, a target far from its point).

> Thirty seconds of checking against hours of a run that was wrong from line 1.

# THE STRUCTURE IN VISIT




The leaf of IB/03 as VisIt draws it from `lag_data.visit`: the `.spring` file as lines, at $t = 0$ and $t = 2$ s, over the vorticity (IBAMR 0.19.0).

# THE STRUCTURE IN VISIT: WHAT IS DRAWN

- IBAMR's Silo writer (`LSiloDataWriter`) turns the structure into a mesh named `<name>_mesh`: the points are the vertices, and the springs and crosslink springs are its edges. Beams, targets, anchors and masses are not drawn.
- So a structure without springs appears as unconnected points, and a band with a missing closing spring shows its gap.
- The positions written are the current ones, every `viz_dump_interval` steps, so the mesh moves with the structure; VisIt's time slider steps through the frames. The leaf's bend at $t = 2$ s above is the `.spring` lines following the points.
- The same writer gives each structure its own mesh, so two plates (IB/07) are two meshes, `plate2d_left_512_mesh` and `plate2d_rght_512_mesh`, which you add as two plots.
- Chapter 26 does this in the GUI; chapter 28 adds the finite element meshes of IBFE, which are written differently (ExodusII).

## STRUCTURES BUILT IN THE PROGRAM INSTEAD OF IN FILES

- `IBStandardInitializer` is a thin layer over `IBRedundantInitializer`, which lets `main.cpp` generate the points and elements in C++ through callback functions (`registerInitStructureFunction`, `registerInitSpringDataFunction`, and the same for beams, targets, rods and the rest), without any files.
- IBAMR's own `IB/explicit/ex0` does this; so do `adv_diff/ex6`, `ex7` and the `IBLevelSet` examples (0.19.0).
- When it pays: parametric studies where the structure changes with an input parameter (no file to regenerate), structures of millions of points (no file to read on every process), and structures that depend on the grid at run time.
- When it does not: anything you want to look at, share or regenerate without a compiler. Files are easier to inspect, version and plot, and every example in this series uses them.

> The same `IBStandardInitializer` block still names the structure and its level; only the data source changes. Chapter 15 walks through `ex0`'s callbacks next to the file version of the same shape.

# IB2d's files, side by side

| File | IBAMR | IB2d |
|---|---|---|
| `.vertex` | count, then `x y` per line; index = line number from 0 | the same, but the index counts from 1 |
| `.spring` | `i j kappa L [fcn [params]]` | `master slave k L [alpha]`, with an optional nonlinearity exponent `alpha` |
| `.beam` | `i j k kappa [Cx Cy]`: a curvature *vector* | `i j k k_beam C`: a scalar curvature; IB2d also has `.nonInv_beam` (non-invariant beams) |
| `.target` | `i kappa [eta]` | `ID k_target` |
| `.mass` | `i mass kappa` | `ID mass k_mass` |
| others | `.xspring`, `.anchor`, `.rod`, `.director`, `.inst`, `.source` | `.muscle`, `.porous`, `.concentration`, `.geo_connect`, ... |

- The geometry is the same: a structure generated for IB2d runs in IBAMR after the indices are shifted by one and the beam curvature is turned into a vector (and the units checked: IB2d's examples are usually dimensionless).
- MeshmerizeMe (chapter 4f) writes IB2d files from a drawing; Part II, chapter 4a, is the IB2d counterpart of this chapter and gives IB2d's full list.

IB2d formats from the IB2d repository's rubber-band example and generators (https://github.com/nickabattista/IB2d).

# READING AN EXAMPLE FOLDER

| In `examples/IB/03-tethered-beam/` | What it is |
|---|---|
| `leaf_512.vertex` | 103 lines after the count: the leaf's points, top to bottom |
| `leaf_512.spring` | 102 springs: `0 1 3.0600e+04 9.8039e-04`, …, with $K/\Delta s$ for $K = 30$ N and $L = \Delta s$ |
| `leaf_512.beam` | 101 beams: `0 1 2 5.3060e+06`, …, with $EI/\Delta s^3$ for $EI = 5 \times 10^{-3}$ N m$^2$, no curvature |
| `leaf_512.target` | 2 lines: points 50 and 51 with $3.06 \times 10^4$, no damping |
| `generate_mesh2d.m`, `.py` | the scripts that wrote the four files; change `N`, `kappa_spring`, `kappa_beam` or `plate_length` here |
| `input2d` | `structure_names = "leaf_512"` and `level_number = MAX_LEVELS - 1`; the rest of the physics |
| `main.cpp` | the rubber band's program: it never mentions the files |
| `README.md` | what the numbers above mean and what the run shows |

> Every example folder in this series reads this way. The `512` in the names is the finest grid count the files were made for; regenerate with another `N` and the names change with it, so that files for different grids never mix.

# HOMEWORK: YOUR OWN RUBBER BAND

From the 2018 and 2020 courses (Appendix C collects the full set):

1. Copy IB/01 and write your own generator, in any language, that puts down the `.vertex` points and the `.spring` springs of a rubber band.
2. Make the band 50% smaller and deform it initially in a different way (an ellipse the other way, a square, a band with a dent).
3. Run it and look at it in VisIt. The 2018 version asked for two screenshots of the band oscillating; the 2020 version for the generator and the pictures in a shared folder.

> What to check in your files before running: the count equals the number of lines minus one; the indices run from 0; the ring is closed; the spacing is about $\Delta x/2$; the stiffness in the file is $K/\Delta s$ for the $K$ you chose. Then a short run and the log's "read N" lines.

Extension: add a `.beam` file to the band (IB/03's generator shows the loop) and watch the dent smooth out faster.

# RULES OF THUMB

| Rule | Why |
|---|---|
| One generator per structure, kept with the files | the files are derived data; the constants and the geometry live in the script (slide 28) |
| Indices from 0, local to the structure | IBAMR's convention; `.xspring` is the only file with global indices (slide 8) |
| Write every column that matters, even when a default would do | the rest length is required anyway; a stated 0 is clearer than a missing value (slide 11) |
| Stiffnesses as $K/\Delta s$ and $EI/\Delta s^3$ | so that the physics does not change when you refine (slide 15) |
| Point spacing about $\Delta x/2$ | chapter 5; the generators compute it from $N$ |
| Closed curves have as many springs as points | the closing spring (slide 12) |
| Read the log's "read N" lines after every change | missing files are skipped silently (slide 25) |
| A short run, then VisIt, then the long run | every file error appears before step 1 (slides 26, 34) |
| Unix line endings everywhere | the structure files tolerate CRLF, the input file does not (slide 23) |

## SUMMARY

1. A structure is a `.vertex` file (required) plus elastic elements in `.spring`, `.beam`, `.target` and, less often, `.xspring`, `.anchor`, `.mass`, `.rod`/`.director`, `.inst` and `.source`; the input file names the structure and its grid level.
2. Points are numbered from 0 by their line in the `.vertex` file; springs join pairs, beams act on triples through their middle point, targets tether single points to a position that the program may move.
3. The file stiffnesses are $K/\Delta s$ and $EI/\Delta s^3$, so they are rescaled by the generator when the grid changes; the rest length of a spring is required, whatever the header says.
4. The log reports what was read and what was skipped; the reader stops with a precise line number for malformed files but says nothing about a missing optional file or a missing closing spring.
5. Generate, count, plot, run a few steps, look in VisIt, then run.

Next, chapter 7: boundary conditions and the set-up of the fluid box, where the input file's other half is explained.

# WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The format table now covers every file the 0.19.0 initializer reads, including `.xspring`, `.anchor`, `.rod`, `.director`, `.inst` and `.source`, which the 2013–2020 lectures listed partly or not at all, and the `uniform_*`, `enable_*`, `posn_shift`, `length_scale_factor` and `use_file_batons` keys.
- The rest-length statement is corrected: required, not optional (tested), against the header comment that the 2020 lecture quoted.
- The units of the file stiffnesses are stated for the first time ($K/\Delta s$, $EI/\Delta s^3$), with the derivation.
- The comment, line-ending and number rules are tested rather than assumed, and the error messages are quoted from runs.
- The closing-spring bug of the UNC generator (643 springs for 644 points) is explained and fixed in IB/01's generator; the generator excerpts are IB/01's current ones, with a Python version.
- The log's "read N" and "skipping read" lines and the zero-stiffness warnings are new material.
- The resources slide is updated: the Google Code links of 2015 and the class-documentation link of 2020 are dead; the header file is the format reference.
- IB2d's formats are compared in one table; the IB2d chapter (4a) keeps the full list.

# REFERENCES


- IBAMR source, version 0.19.0: `include/ibamr/IBStandardInitializer.h` (the file-format comments), `src/IB/IBStandardInitializer.cpp` (the readers), `src/IB/IBStandardForceGen.cpp` and `include/ibamr/IBSpringForceFunctions.h` (the forces), `src/IB/IBRedundantInitializer.cpp` (structures from code). https://github.com/IBAMR/IBAMR
- IBAMR documentation index: https://ibamr.github.io/docs; wiki and FAQ: https://github.com/IBAMR/IBAMR/wiki; users' group: https://groups.google.com/g/ibamr-users; discussions: https://github.com/IBAMR/IBAMR/discussions.
- C. S. Peskin. The immersed boundary method. *Acta Numerica* 11:479–517, 2002. https://math.nyu.edu/~peskin/
- N. A. Battista, W. C. Strickland and L. A. Miller. IB2d: a Python and MATLAB implementation of the immersed boundary method. *Bioinspiration & Biomimetics* 12:036003, 2017. https://github.com/nickabattista/IB2d
- D. M. Senter et al. A semi-automated finite difference mesh creation method for use with immersed boundary software IB2d and IBAMR. *Bioinspiration & Biomimetics* 16:016008, 2021.
- The tests behind slides 23 and 26: `wsl/final/ch6_errors.sh` in the working folder of these notes, run with IBAMR 0.19.0 on the rubber band of IB/01.
- Sources: *IBAMR Tutorial: Getting Started* (2013–2021); *MATH 769 Lecture 5: IBAMR Input Files* (2015); *3 – IBAMR Associated Files* (2018–2019); *IBAMR Lecture 3* (2020, edited 2024); the lab's 2012 ReadMe and 2013 input-visualization decks.